SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 8: Report

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – MAR-2026

Contents

1. Describe the function you wrote to construct DL models dynamically. How does it allow you to swap between LSTM, RNN, and GRU layers without rewriting the whole training script? 3

2. What is “Multistep Prediction”? How does the output layer of your model change when you want to predict a sequence of k future days instead of just tomorrow? 3

3. Explain “Multivariate Prediction.” When predicting the ‘Close’ price, how do you structure the input array to include other time series like ‘Trading Volume’? 3

1. In stock prediction, why is Mean Squared Error (MSE) or Mean Absolute Error (MAE) often preferred over simple "Accuracy" used in classification?

I wrote a function called *build_dynamic_dl_model()* to build the deep learning model dynamically. Instead of hard-coding only one model type, the function takes a parameter called model_type. Inside the function, I use a mapping that connects the string value to the correct recurrent layer type: LSTM, GRU, or SimpleRNN. After choosing the layer type, the function builds the model using the same overall structure, including the input shape, number of layers, number of units, dropout rate, and output layer. Because of this design, I can switch between LSTM, RNN, and GRU simply by changing one argument, while keeping the rest of the training script unchanged. The data loading, training, and prediction pipeline do not need to be rewritten.

2. How does the choice of "Batch Size" and "Number of Epochs" impact the convergence of your stock price model?

- Batch size
    - Smaller batch (e.g., 16) updates weights more frequently per epoch → gradient is noisier but can generalize well.
    - Larger batch (e.g., 64) uses smoother gradients and can run faster per step, but may converge differently and sometimes generalize worse.
    - In your runs (Batch 16, Epoch 20), test MAE is around ~4 (3.96–4.06), showing reasonable performance but with some variability run-to-run.
- Epochs
    - More epochs usually reduces training loss (better fitting) up to a point.
    - Too many epochs can cause overfitting (training improves while test error worsens).
    - You decide “good convergence” by watching loss curve + test MAE/MSE, not training loss alone.

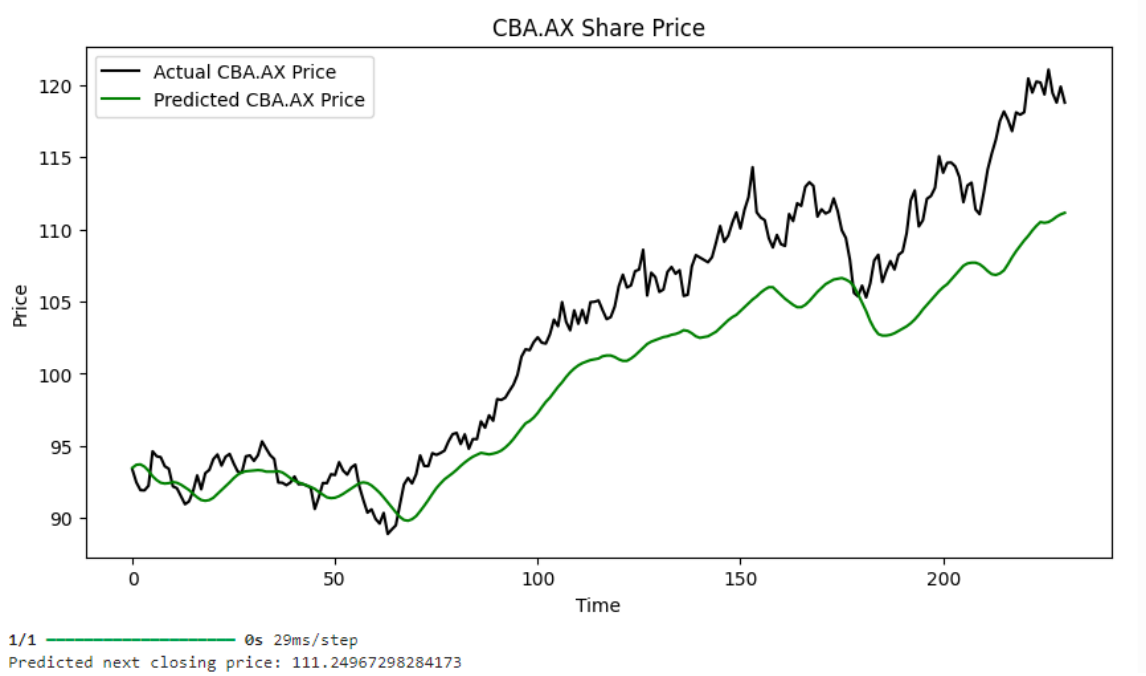

Figure 1: CBA.AX – Actual vs Predicted Closing Price (LSTM, Batch 16, Epoch 20)prices after the end of the available dataset.

Model	***Batch***	***Epochs***	***Train time (sec)***	***Test MAE***	***Test MSE***	***Test RMSE***
LSTM	*16*	*20*	*54.35*	*3.9592*	*23.1255*	*4.8089*
LSTM	*16*	*20*	*104.06*	*4.0625*	*25.1490*	*5.0149*
LSTM	*32*	*30*	*54.35*	*3.95919*	*23.12550*	*4.80890*
LTSM	*64*	*50*	*248.91*	*4.08796*	*24.62739*	*4.96260*

Figure 2: Hyperparameter Evaluation – Batch/Epoch vs MAE/MSE (CBA.AX)

3. Compare the performance of an LSTM network vs. a GRU network for your specific stock. Which one was faster to train, and which was more accurate?

Experiment context (same dataset/stock):

- Stock: CBA.AX
- Features used: Close, Volume, High, Low
- Lookback window: 60 days
- Goal: Compare training speed and prediction performance based on the results in my Week 7 notebook.

A. Training Speed (Which trained faster?)

Model	Task / Output	Epochs	Batch	Trainable Params	Observed Training Time
LSTM	Single-step (predict 1 next Close)	30	32	51,451	~140s total (~4.67s/epoch)
GRU	Multi-step (predict next 5 days)	20	32	39,255	~94s total (~4.70s/epoch)

Conclusion (Speed): GRU trained faster overall in my notebook run (fewer parameters + fewer epochs → shorter total training time).

B. Prediction Performance (Which was more accurate?)

In my Week 7 notebook, the two models were not evaluated using exactly the same test metric setup:

- LSTM was trained for single-step prediction, but I did not compute test MAE/MSE for the full test set in Week 7.
- GRU was trained for 5-step prediction, and I printed/visualized predictions vs actual values for the first few test days.

Evidence available from my notebook outputs:

- Training loss (scaled MSE) at the end of training:
    - LSTM final loss ≈ 0.0048
    - GRU final loss ≈ 0.0069

Because lower loss indicates better fit (on the same scaled loss function), this suggests the LSTM achieved better model fit in my run.

Additionally, for GRU, I verified a small test-window (first few predicted days), which showed reasonable errors, but it is not directly comparable to LSTM unless both are evaluated on the same horizon and same metric over the full test set.

Conclusion (Accuracy): Based on the results recorded in Week 7, LSTM appears more accurate (lower final loss), while GRU was faster to train.
A fully fair comparison requires computing MAE/MSE for both models on the same test set and the same prediction horizon.